

Simulazione di un attacco UDP Flood

1. Obiettivo dell'Esercitazione

L'obiettivo è sviluppare e analizzare uno script in linguaggio Python in grado di simulare un attacco **DoS (Denial of Service)**. Nello specifico, si intende realizzare un **UDP Flood**, inviando una grande quantità di pacchetti verso un bersaglio designato per analizzarne il comportamento e l'impatto sulle risorse di sistema.

2. Cenni Teorici

- **Protocollo UDP:** L'User Datagram Protocol (UDP) è un protocollo di trasporto "connectionless" (senza connessione). A differenza del TCP, non prevede una fase di handshake (stabilimento della connessione) né garanzie di consegna.
- **Vantaggio offensivo:** La mancanza di handshake rende l'UDP estremamente veloce e ideale per generare traffico massivo.
- **Scopo dell'attacco:** Saturare la banda di rete o le risorse di elaborazione (CPU) della vittima (windows XP)

3. Descrizione del Programma

Lo script è stato realizzato con l'aiuto di Gemini, in Python, utilizzando le librerie standard. Di seguito i passaggi logici fondamentali:

- **Librerie utilizzate:**
 - socket: Per la gestione delle connessioni di rete.
 - random: Per generare il contenuto casuale dei pacchetti.
- **Configurazione del Socket:** Il cuore del programma risiede nell'istruzione: `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`
 - AF_INET: Indica l'utilizzo di indirizzi IPv4.
 - SOCK_DGRAM: Specifica l'utilizzo del protocollo UDP (Datagram).
- **Creazione del Payload:** Per simulare traffico reale e saturare la banda, viene generato un pacchetto di dimensioni fisse pari a **1 KB (1024 bytes)** contenente dati casuali. Il payload viene generato *una sola volta* prima del ciclo di invio.
- **Ciclo di invio:** Il programma richiede all'utente l'IP target, la porta target e il numero di pacchetti. Successivamente, entra in un ciclo for che utilizza il metodo `sendto()` per inviare i pacchetti all'indirizzo specificato senza attendere risposta.

```
udp_flood2.py X
udp_flood2.py > ...
1  import socket
2  import random
3
4  # --- 1. Acquisizione Input Utente ---
5  target_ip = input("Inserisci IP Target: ")
6  target_port = int(input("Inserisci Porta UDP Target: "))
7  n_pacchetti = int(input("Inserisci numero di pacchetti da inviare: "))
8
9  # --- 2. Creazione del Socket ---
10 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
11
12 # --- 3. Preparazione del Pacchetto ---
13 payload = random.randbytes(1024)
14
15 print(f"\n[+] Inizio invio di {n_pacchetti} pacchetti verso {target_ip}:{target_port}...")
16
17 # --- 4. Ciclo di Invio (Flooding) ---
18 for i in range(n_pacchetti):
19     # sendto invia il pacchetto all'indirizzo (IP, Porta)
20     client_socket.sendto(payload, (target_ip, target_port))
21
22 # --- 5. Chiusura ---
23 print("[+] Operazione completata.")
24 client_socket.close()
```

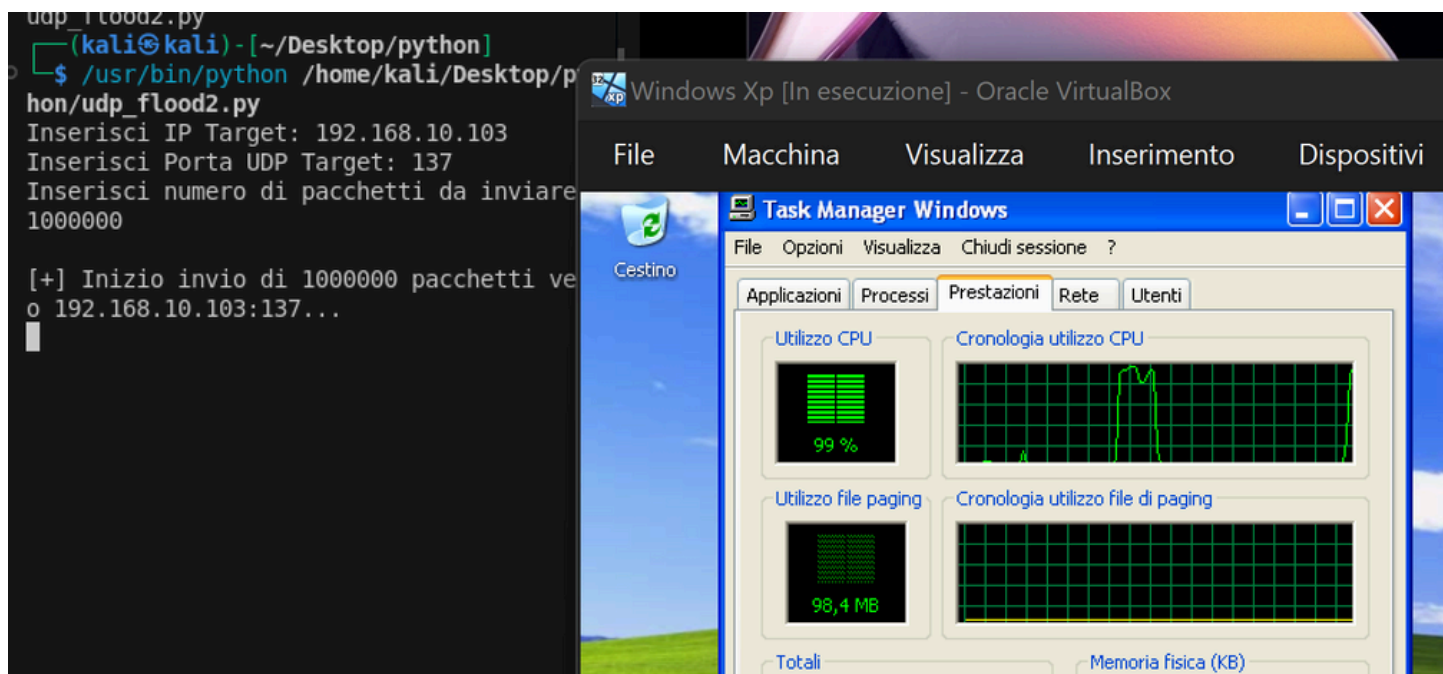
4. Esecuzione e Test

Il test è stato condotto in un ambiente di laboratorio virtuale isolato con la seguente configurazione:

- **Macchina Attaccante:** Kali Linux
- **Macchina Vittima:** Windows XP
- **Strumenti di Monitoraggio:** Task Manager

Procedura:

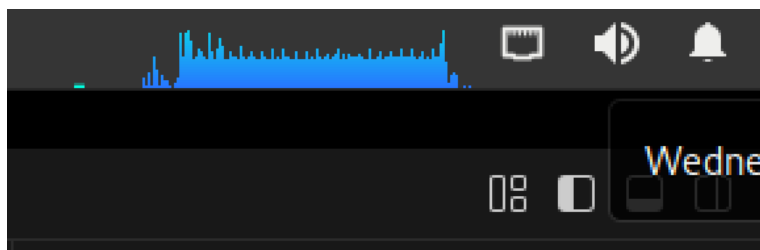
1. Avvio dello script Python sulla macchina attaccante.
2. Inserimento dei parametri (IP Target: 192.168.10.103, Porta: 137).
***PS:** la porta udp aperta è stata precedentemente individuata con il seguente comando*
`nmap -sS -sU -p80-445 192.168.50.103`
3. Esecuzione dell'attacco per un numero prefissato di pacchetti (1.000.000).



5. Osservazioni

Durante l'esecuzione del test sono emersi i seguenti comportamenti:

- **Lato Vittima:** Si è osservato un aumento critico dell'utilizzo della CPU sulla macchina target. Tale fenomeno non è causato dalla complessità dei dati ricevuti (che sono semplici byte casuali), ma dall'elevatissima frequenza. Ogni pacchetto che raggiunge l'interfaccia di rete genera una richiesta di interruzione verso il processore e la capacità di calcolo viene quasi interamente assorbita dalla gestione dell'input di rete.
- **Lato Attaccante:** Anche la macchina attaccante ha mostrato un elevato utilizzo della CPU, per l'elevato numero di sendto eseguiti.



6. Conclusioni

L'esercitazione ha permesso di comprendere la semplicità con cui è possibile generare traffico UDP utilizzando poche righe di codice Python. È emerso come l'UDP, privo di meccanismi di controllo flusso, permetta di inviare dati alla massima velocità possibile, trasferendo il carico di elaborazione sulla vittima. Tuttavia, l'esperimento ha anche evidenziato che un attacco DoS richiede risorse significative anche da parte dell'attaccante ("rinculo digitale").